

Sorting algorithms

Updated 6/18/1015

Most of the time, we will use "canned" sorting algorithms when we do data science.

But, it is important to be aware of some common algorithms, so we will look at a couple sorting algorithms

We also need to understand how to characterize the run time or efficiency of an algorithm

Bubble sort is from:

<https://www.geeksforgeeks.org/python-program-for-bubble-sort/>

Bubble Sort

This was invented by Edward Friend in 1956.

It is not the best available sort, but it is taught in most intro to programming courses, so we need to see it.

```
In [1]: def bubble_sort(arr):

    nouter=len(arr)

    # Outer loop to iterate through the list n times, starting at the 2nd to
    for n in range(nouter - 1, 0, -1):

        # Inner loop to compare adjacent elements
        # start at zero and go up to

        for i in range(n):
            if arr[i] > arr[i + 1]:
                # Swap elements if they are in the wrong order
                arr[i], arr[i + 1] = arr[i + 1], arr[i]

    # the outer loop starts with n=length-1
    # the inner loop then moves the largest item it finds to the end of the list
    # the outer loop then runs at n=length-2, and the second largest item is mov

    # Sample list to be sorted
    arr = [39, 12, 18, 85, 72, 10, 2, 18]
    print("Unsorted list is:")
    print(arr)
```

```
bubble_sort(arr)

print("Sorted list is:")
print(arr)
```

Unsorted list is:

[39, 12, 18, 85, 72, 10, 2, 18]

Sorted list is:

[2, 10, 12, 18, 18, 39, 72, 85]

Question/Action

Only one simple change on one line of the function is needed to make it sort in descending form

Copy and paste, then edit to bubble_sort_descending, show that this works.

Dont' alter the loop directions...

```
In [6]: def bubble_sort_descending(arr):

    nouter=len(arr)

    # Outer loop to iterate through the list n times, starting at the 2nd to
    for n in range(nouter - 1, 0, -1):

        # Inner loop to compare adjacent elements
        # start at zero and go up to

        for i in range(n):
            if arr[i+1] > arr[i]:
                # Swap elements if they are in the wrong order
                arr[i], arr[i + 1] = arr[i + 1], arr[i]

    # the outer loop starts with n=length-1
    # the inner loop then moves the largest item it finds to the end of the list
    # the outer loop then runs at n=length-2, and the second largest item is mov

    # Sample list to be sorted
    arr = [39, 12, 18, 85, 72, 10, 2, 18]
    print("Unsorted list is:")
    print(arr)
    print(len(arr))

    bubble_sort_descending(arr)

    print("Sorted list is:")
    print(arr)
```

Unsorted list is:
[39, 12, 18, 85, 72, 10, 2, 18]
8
Sorted list is:
[85, 72, 39, 18, 18, 12, 10, 2]

Question/Action

Explain what the first "pass" through the inner loop will do to the list

How many steps are there in the first time through the inner loop?

Why is the length of the inner loop reduced after the each time it runs? Why don't we go all the way to the end of the list each time?

During the first full pass of the inner loop, adjacent elements are compared and swapped if needed. By the end of the first pass, the largest number (85) has bubbled to the end of the list. [12, 18, 39, 72, 10, 2, 18, 85]

There are 7 steps in the first pass through the inner loop because a list of 8 elements requires 7 adjacent comparisons.

The inner loop is reduced after each pass because the largest element is already placed at the end of the list. Since that value is in its correct position, there is no need to compare it again in future passes.

Timing our code

How fast is our code? How do we describe this?

Let's use %timeit to time how long a bubble sort takes at some different array lengths

%timeit times the operation of one function, so to use %timeit, we need to be working with a function

The number of repetitions of the loop is $n*(n-1)$, so the run time should be proportional to n^2

The speed of the algorithm is said to be order of n^2 , written as $O(n^2)$

In Computer science, we really want to know the order of the algorithms we use

```
In [7]: import random
random_numbers = [random.random() for _ in range(50)]
```

In [8]: *#50 values in random numbers*

```
%timeit bubble_sort(random_numbers)
```

13.5 μ s $\pm$ 571 ns per loop (mean $\pm$ std. dev. of 7 runs, 100,000 loops each)

In [9]: `random_numbers = [random.random() for _ in range(500)]`

```
%timeit bubble_sort(random_numbers)
```

1.51 ms $\pm$ 53.6 μ s per loop (mean $\pm$ std. dev. of 7 runs, 1,000 loops each)

In [10]: `random_numbers = [random.random() for _ in range(2000)]`

```
%timeit bubble_sort(random_numbers)
```

31.1 ms $\pm$ 781 μ s per loop (mean $\pm$ std. dev. of 7 runs, 10 loops each)

In [11]: `random_numbers = [random.random() for _ in range(5000)]`

```
%timeit bubble_sort(random_numbers)
```

223 ms $\pm$ 3.04 ms per loop (mean $\pm$ std. dev. of 7 runs, 1 loop each)

In [25]: `import matplotlib.pyplot as plt`

```
import numpy as np
```

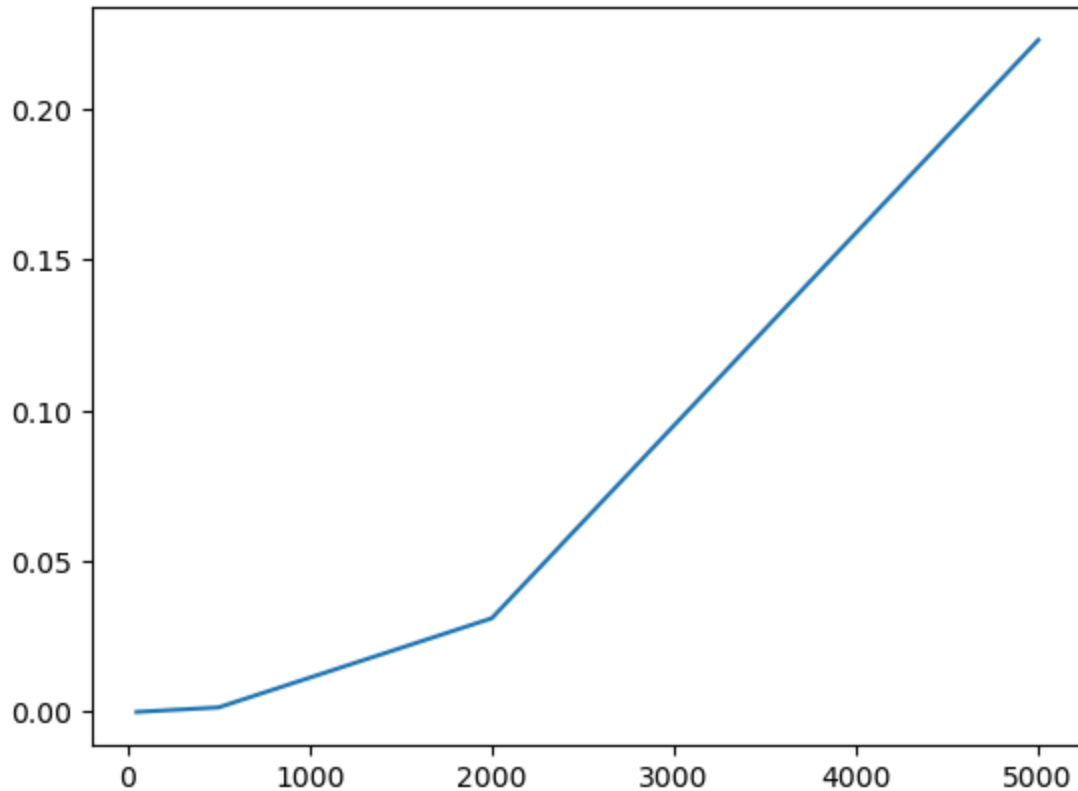
*# note: the values included in time are the values I measured on my computer
edit the list time so the times in the list are the ones you measured*

```
time=[13.5e-6, 1.51e-3,31.1e-3,223e-3]
```

```
n=[50,500,2000,5000]
```

```
plt.plot(n,time)
```

Out[25]: [`<matplotlib.lines.Line2D at 0x1355107d0>`]



An Alternative to %timeit

The %timeit function in Jupyter lab is a bit limited, it has to work with a function

We can import the datetime library and used it to find the elapsed time

```
In [13]: from datetime import datetime

random_numbers = [random.random() for _ in range(500)]

start_time = datetime.now()
bubble_sort(random_numbers)
end_time = datetime.now()

elapsed_time = end_time - start_time
print(f'Cell ran in {elapsed_time}')
```

Cell ran in 0:00:00.009475

Mergesort

A better sorting algorithm

The *mergesort* is a faster, more efficient way to sort.

```
In [15]: import math    # needed for the floor() operation

def merge(left, right):
    result = []
    i, j = 0, 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result += left[i:]
    result += right[j:]
    return result

def mergesort(list):
    if len(list) < 2:
        return list
    middle = math.floor(len(list) / 2)
    left = mergesort(list[:middle])
    right = mergesort(list[middle:])
    return merge(left, right)
```

```
In [16]: mergesort(arr)
```

```
Out[16]: [2, 10, 12, 18, 18, 39, 72, 85]
```

How does the Mergesort work?

Note this implementation is recursive.

If the length of a list is 1, we just return it

Other wise

1.) find the midpoint 2.) Mergesort the left half 3.) Mergesort the right half 3.) "Merge" the two sorted halves of the two lists, in the values in the two lists are combined back into a single list, always by adding the smallest value in either the two lists into the single merged list.

How fast should mergesort be?

This can get complicated, but it appears that mergesort is faster than

$O(n \log n + n + O(\log n))$

the lead term here is $n \log(n)$, which is less than n^2

mergesort is a bit unpredictable in its timing, it can vary greatly with the amount of order already present

```
In [17]: random_numbers = [random.random() for _ in range(500)]
%timeit mergesort(random_numbers)
```

277 μ s $\pm$ 3.79 μ s per loop (mean $\pm$ std. dev. of 7 runs, 1,000 loops each)

```
In [18]: random_numbers = [random.random() for _ in range(5000)]
%timeit mergesort(random_numbers)
```

3.72 ms $\pm$ 7.48 μ s per loop (mean $\pm$ std. dev. of 7 runs, 100 loops each)

Question/Action

Repeat the plot of the dependence of execution time on the number of items sorted that we created for the BubbleSort, but now for the mergesort

Generate a series of tests of the time to run a mergesort over the same range of n values we used for the bubblesort and set up lists of n_{merge} and t_{merger} .

Plot the time vs n for the bubblesort and the mergesort on a single graph.

```
In [20]: # Generate a series of tests
random_numbers = [random.random() for _ in range(50)]
%timeit mergesort(random_numbers)
```

20 μ s $\pm$ 137 ns per loop (mean $\pm$ std. dev. of 7 runs, 10,000 loops each)

```
In [21]: random_numbers = [random.random() for _ in range(500)]
%timeit mergesort(random_numbers)
```

274 μ s $\pm$ 1.34 μ s per loop (mean $\pm$ std. dev. of 7 runs, 1,000 loops each)

```
In [22]: random_numbers = [random.random() for _ in range(2000)]
%timeit mergesort(random_numbers)
```

1.32 ms $\pm$ 5.09 μ s per loop (mean $\pm$ std. dev. of 7 runs, 1,000 loops each)

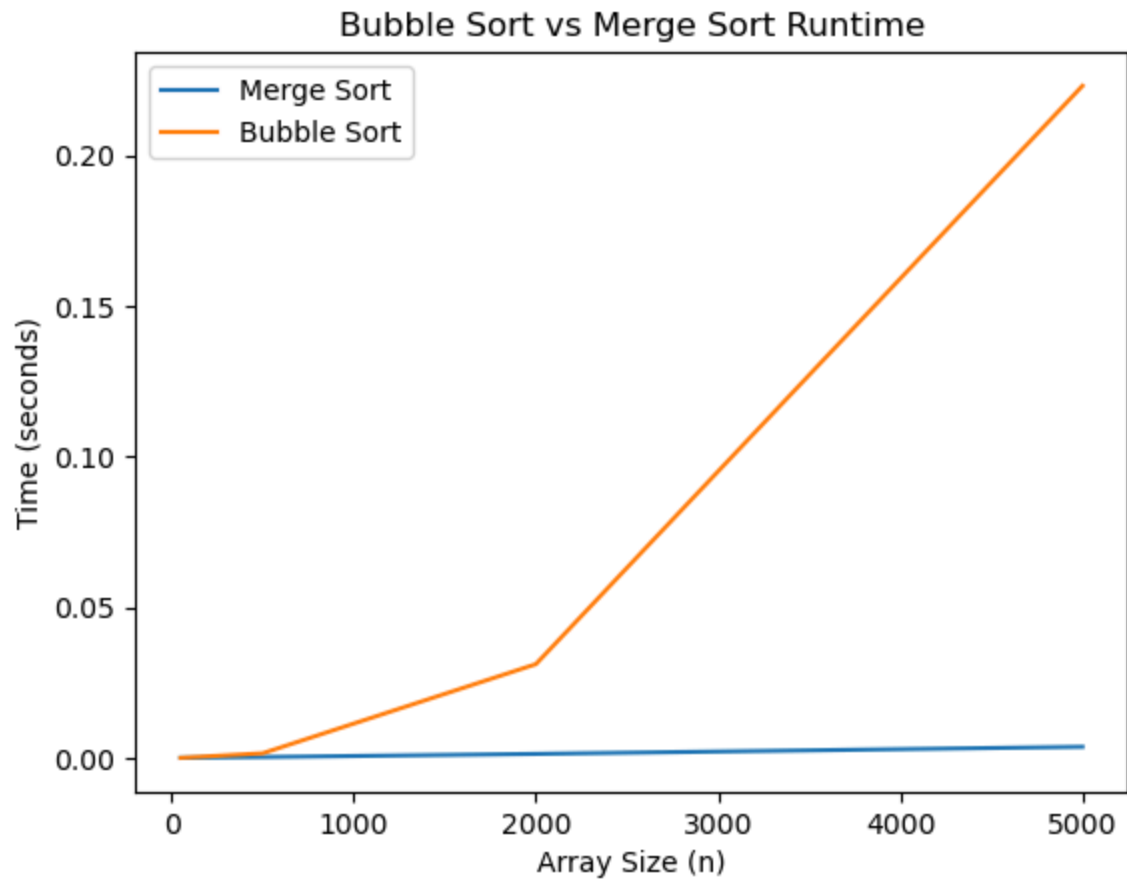
```
In [23]: random_numbers = [random.random() for _ in range(5000)]
%timeit mergesort(random_numbers)
```

3.67 ms $\pm$ 7.51 μ s per loop (mean $\pm$ std. dev. of 7 runs, 100 loops each)

```
In [34]: t_merger = [20e-6, 274e-6, 1.32e-3, 3.67e-3]
n_merge = [50, 500, 2000, 5000]

plt.plot(n_merge, t_merger, label = "Merge Sort")
plt.plot(n, time, label = "Bubble Sort")
plt.xlabel("Array Size (n)")
plt.ylabel("Time (seconds)")
plt.title("Bubble Sort vs Merge Sort Runtime")
```

```
plt.legend()  
plt.show()
```



In []: